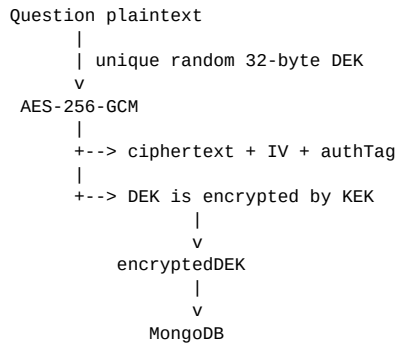


Full Secure Question Bank Implementation

Stack: Node.js + Express + MongoDB + AES-256-GCM + envelope encryption. This is a prototype/learning implementation. Production examination infrastructure should use a managed KMS/HSM, hardened authentication, auditing, secure deployment and independent security review.

1. Exactly where DEK and KEK are stored



KEK:
NEVER store in the questions collection.
NEVER hard-code it in JavaScript.
Prototype: environment/secret store.
Production: KMS/HSM.

DEK:
Plain DEK exists only during encryption/decryption.
MongoDB stores ONLY encryptedDEK + its IV + authTag.

Important: MongoDB therefore contains the encrypted question and an encrypted copy of its DEK. The plaintext DEK is not persisted. The KEK protects the DEKs and should be outside the database.

2. Project structure

```
secure-question-bank/
  server.js
  .env
  package.json
  config/
    db.js
  models/
    Question.js
  services/
    cryptoService.js
  controllers/
    questionController.js
  routes/
    questionRoutes.js
  middleware/
    auth.js          # add real auth before production
  scripts/
    generateKek.js
    seedQuestions.js
```

3. Install

```
mkdir secure-question-bank
cd secure-question-bank
npm init -y
npm install express mongoose dotenv
npm install --save-dev nodemon
```

4. Generate the KEK

Generate the KEK with a cryptographically secure random generator. Do not type a human password as the KEK.

```
// scripts/generateKek.js
const crypto = require("crypto");

const kek = crypto.randomBytes(32);

console.log("KEK_BASE64=");
```

```
console.log(kek.toString("base64"));

node scripts/generateKek.js
```

Copy the resulting base64 value into a protected secret store. For a local prototype, .env is acceptable. Do not commit .env to Git.

5. .env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/secure_question_bank
KEK_BASE64=PASTE_THE_32_BYTE_BASE64_VALUE_HERE

.gitignore
node_modules/
.env
```

6. MongoDB connection

```
// config/db.js
const mongoose = require("mongoose");

async function connectDB() {
  await mongoose.connect(process.env.MONGODB_URI);
  console.log("MongoDB connected");
}

module.exports = connectDB;
```

7. MongoDB Question model

```
// models/Question.js
const mongoose = require("mongoose");

const questionSchema = new mongoose.Schema(
  {
    ciphertext: { type: String, required: true },
    iv: { type: String, required: true },
    authTag: { type: String, required: true },

    // DEK is encrypted; plaintext DEK is NOT stored.
    encryptedDEK: { type: String, required: true },
    encryptedDEKiv: { type: String, required: true },
    encryptedDEKAuthTag: { type: String, required: true },

    subject: { type: String, required: true },
    topic: { type: String, required: true },
    difficulty: {
      type: String,
      enum: ["Easy", "Medium", "Hard"],
      required: true
    },
    marks: { type: Number, required: true }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Question", questionSchema);
```

8. Complete AES-256-GCM + key-generation service

This is the core. Each call to encryptQuestion() generates a fresh 32-byte DEK. A fresh 12-byte IV is generated for every AES-GCM encryption operation.

```
// services/cryptoService.js
const crypto = require("crypto");

const ALGORITHM = "aes-256-gcm";
const KEY_BYTES = 32;
const IV_BYTES = 12;

function getKEK() {
  const b64 = process.env.KEK_BASE64;

  if (!b64) {
    throw new Error("KEK_BASE64 is missing");
  }

  const key = Buffer.from(b64, "base64");

  if (key.length !== KEY_BYTES) {
```

```

    throw new Error("KEK must be exactly 32 bytes");
  }

  return key;
}

function generateDEK() {
  // 32 random bytes = 256-bit DEK
  return crypto.randomBytes(KEY_BYTES);
}

function aesGcmEncrypt(plaintext, key) {
  const iv = crypto.randomBytes(IV_BYTES);

  const cipher = crypto.createCipheriv(
    ALGORITHM,
    key,
    iv
  );

  const ciphertext = Buffer.concat([
    cipher.update(plaintext, "utf8"),
    cipher.final()
  ]);

  const authTag = cipher.getAuthTag();

  return {
    ciphertext: ciphertext.toString("base64"),
    iv: iv.toString("base64"),
    authTag: authTag.toString("base64")
  };
}

function aesGcmDecrypt(data, key) {
  const decipher = crypto.createDecipheriv(
    ALGORITHM,
    key,
    Buffer.from(data.iv, "base64")
  );

  decipher.setAuthTag(
    Buffer.from(data.authTag, "base64")
  );

  const plaintext = Buffer.concat([
    decipher.update(
      Buffer.from(data.ciphertext, "base64")
    ),
    decipher.final()
  ]);

  return plaintext.toString("utf8");
}

function encryptDEK(dek) {
  const result = aesGcmEncrypt(
    dek.toString("base64"),
    getKEK()
  );

  return result;
}

function decryptDEK(record) {
  const dekBase64 = aesGcmDecrypt(
    {
      ciphertext: record.encryptedDEK,
      iv: record.encryptedDEKiv,
      authTag: record.encryptedDEKAuthTag
    },
    getKEK()
  );

  return Buffer.from(dekBase64, "base64");
}

function encryptQuestion(question) {
  const dek = generateDEK();

  const encryptedQuestion =
    aesGcmEncrypt(question, dek);

  const encryptedDek =
    encryptDEK(dek);

  // Remove plaintext DEK from this Buffer after use.
  dek.fill(0);
}

```

```

return {
  ...encryptedQuestion,

  encryptedDEK: encryptedDek.ciphertext,
  encryptedDEKiv: encryptedDek.iv,
  encryptedDEKAuthTag: encryptedDek.authTag
};
}

function decryptQuestion(record) {
  const dek = decryptDEK(record);

  try {
    return aesGcmDecrypt(
      {
        ciphertext: record.ciphertext,
        iv: record.iv,
        authTag: record.authTag
      },
      dek
    );
  } finally {
    dek.fill(0);
  }
}

module.exports = {
  encryptQuestion,
  decryptQuestion
};

```

9. Controller: encrypt every incoming question

```

// controllers/questionController.js
const Question = require("../models/Question");
const {
  encryptQuestion,
  decryptQuestion
} = require("../services/cryptoService");

async function createQuestion(req, res) {
  try {
    const {
      question,
      subject,
      topic,
      difficulty,
      marks
    } = req.body;

    if (
      typeof question !== "string" ||
      !question.trim() ||
      !subject ||
      !topic ||
      !difficulty ||
      !Number.isInteger(marks)
    ) {
      return res.status(400).json({
        message: "Invalid question data"
      });
    }

    // UNIQUE DEK is generated inside this call.
    const encrypted = encryptQuestion(question);

    const saved = await Question.create({
      ...encrypted,
      subject,
      topic,
      difficulty,
      marks
    });

    return res.status(201).json({
      message: "Question encrypted and stored",
      questionId: saved._id
    });
  } catch (err) {
    console.error(err);
    return res.status(500).json({
      message: "Failed to create question"
    });
  }
}

async function getQuestion(req, res) {

```

```

try {
  const record = await Question.findById(req.params.id);

  if (!record) {
    return res.status(404).json({
      message: "Question not found"
    });
  }

  // IMPORTANT:
  // Real system must perform authentication,
  // role checks, exam authorization, time checks,
  // and audit logging BEFORE decryption.
  const plaintext = decryptQuestion(record);

  return res.json({
    id: record._id,
    question: plaintext,
    subject: record.subject,
    topic: record.topic,
    difficulty: record.difficulty,
    marks: record.marks
  });
} catch (err) {
  console.error(err);
  return res.status(500).json({
    message: "Decryption failed"
  });
}

module.exports = {
  createQuestion,
  getQuestion
};

```

10. Routes

```

// routes/questionRoutes.js
const express = require("express");
const router = express.Router();

const {
  createQuestion,
  getQuestion
} = require("../controllers/questionController");

router.post("/", createQuestion);
router.get("/:id", getQuestion);

module.exports = router;

```

11. Main Express server

```

// server.js
require("dotenv").config();

const express = require("express");
const connectDB = require("../config/db");
const questionRoutes = require("../routes/questionRoutes");

const app = express();

app.use(express.json({ limit: "50kb" }));

app.use("/api/questions", questionRoutes);

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((err) => {
    console.error("Database connection failed:", err);
    process.exit(1);
  });

```

12. How ALL questions get encrypted

You do not need to manually encrypt Q1, Q2, Q3. Every POST request to /api/questions invokes encryptQuestion(). That function generates a new random DEK for that question.

```
Q1 request
-> generate DEK-1
-> AES-256-GCM(Q1, DEK-1)
-> encrypt DEK-1 with KEK
-> MongoDB
```

```
Q2 request
-> generate DEK-2
-> AES-256-GCM(Q2, DEK-2)
-> encrypt DEK-2 with KEK
-> MongoDB
```

```
Q3 request
-> generate DEK-3
-> AES-256-GCM(Q3, DEK-3)
-> encrypt DEK-3 with KEK
-> MongoDB
```

So 5,000 questions simply result in 5,000 encryption operations and 5,000 unique DEKs. The same KEK protects the DEKs.

13. Test with Postman/curl

```
POST http://localhost:5000/api/questions
Content-Type: application/json
```

```
{
  "question": "Explain database normalization.",
  "subject": "DBMS",
  "topic": "Normalization",
  "difficulty": "Medium",
  "marks": 5
}
```

```
curl -X POST http://localhost:5000/api/questions \
-H "Content-Type: application/json" \
-d '{
  "question": "Explain database normalization.",
  "subject": "DBMS",
  "topic": "Normalization",
  "difficulty": "Medium",
  "marks": 5
}'
```

14. What MongoDB contains

```
{
  "_id": "...",
  "ciphertext": "BASE64_CIPHertext",
  "iv": "BASE64_12_BYTE_IV",
  "authTag": "BASE64_AUTH_TAG",

  "encryptedDEK": "BASE64_ENCRYPTED_DEK",
  "encryptedDEKiv": "BASE64_IV_FOR_DEK",
  "encryptedDEKAuthTag": "BASE64_TAG_FOR_DEK",

  "subject": "DBMS",
  "topic": "Normalization",
  "difficulty": "Medium",
  "marks": 5
}
```

Notice: there is no field called plaintextQuestion and no field containing the plaintext DEK.

15. What happens during decryption

```
MongoDB record
|
v
decrypt encryptedDEK using KEK
|
v
recover DEK temporarily in application memory
|
v
AES-256-GCM decrypt ciphertext using DEK
|
v
plaintext question
|
v
```

```
use only for the authorized operation
|
v
zero the DEK buffer
```

16. Integrating 4-of-5 authorization

Do NOT make the GET endpoint above public in a real system. Before `decryptQuestion(record)`, your authorization service should verify that the current exam operation has four valid approvals out of five.

```
// Conceptual controller gate

if (!isAuthenticated(req)) {
  return res.status(401).json({ message: "Authentication required" });
}

if (!isAuthorizedForExam(req.user, examId)) {
  return res.status(403).json({ message: "Not authorized" });
}

if (!isWithinExamTimeWindow(exam)) {
  return res.status(403).json({ message: "Outside time window" });
}

if (approvalCount(examId) < 4) {
  return res.status(403).json({
    message: "4-of-5 authorization required"
  });
}

// ONLY NOW decrypt the selected question.
const plaintext = decryptQuestion(record);
```

17. Recommended complete ZEEA flow

```
Question setter
|
v
POST question
|
v
Authentication + role authorization
|
v
Generate unique DEK
|
v
AES-256-GCM encrypt question
|
v
Encrypt DEK with KEK/KMS
|
v
MongoDB encrypted question bank
|
| exam creation
v
Blueprint
|
v
Select questions by metadata
|
v
4-of-5 authorization
|
v
Decrypt ONLY selected questions
|
v
Generate final paper
|
v
Encrypt final paper
|
v
Scheduled/time-limited access
|
v
Controlled release + audit log
```

18. Where the KEK should live

Development:

```

.env
-> never commit to Git

Better deployment:
  Secret Manager / KMS

High-security examination environment:
  HSM/KMS
  |
  +-- KEK remains inside protected key-management boundary
  +-- application requests cryptographic operations
  +-- plaintext KEK is not exposed to normal application code

```

For your final-year prototype, .env can demonstrate the concept. For an actual examination system, use a KMS/HSM rather than treating .env as a secure key vault.

19. Where the DEK should live

```

At encryption:
  generated in application memory
  -> encrypt question
  -> encrypt DEK with KEK
  -> plaintext DEK cleared

At rest:
  MongoDB stores encryptedDEK only

At decryption:
  encryptedDEK retrieved
  -> KEK decrypts it
  -> DEK temporarily exists in memory
  -> question decrypted
  -> DEK buffer cleared

```

20. Security improvements before production

- Add JWT/session authentication and MFA for privileged users.
- Add role-based access control: setter, reviewer, exam authority, auditor.
- Add 4-of-5 threshold authorization with independently authenticated approvers.
- Use a KMS/HSM for KEK management; never put production KEKs in source code.
- Use HTTPS/TLS and secure cookies/tokens.
- Add rate limiting, request validation and security headers.
- Never log plaintext questions, DEKs, KEKs or tokens.
- Add immutable/append-only audit logging where appropriate.
- Use server-side trusted time for exam release and expiry.
- Protect database backups and KMS/HSM access separately.
- Plan key rotation and compromise recovery.
- Perform a professional security review before real exam deployment.

21. Critical clarification about the 4-of-5 keys

The per-question DEK is NOT the same thing as the five people's authorization shares. Keep these concepts separate. The DEK encrypts the question. The multi-party threshold mechanism authorizes the sensitive exam-generation/release operation. This separation keeps key management manageable.

22. Run the project

```

# Start MongoDB, then:
node server.js

# Create questions with:
POST /api/questions

# Read a question only after you add the real
# authentication/authorization/time/approval gates:

```

GET /api/questions/:id

Result: every question gets a different randomly generated 256-bit DEK; each question is encrypted with AES-256-GCM; MongoDB stores ciphertext and encrypted DEK material; the KEK stays outside MongoDB; and the final ZEEA layer can place 4-of-5 authorization and time-based release before decryption.

This PDF provides a complete learning baseline, not a production-ready secure examination system. Cryptographic implementation and key-management choices should be independently reviewed before real-world deployment.